

SMART INDIA HACKATHON 2026 · SIH26099

ALADIN

Remaining Risks & Remediation Plan

National Unified Material Master — “One Nation, One Material Code”

Purpose: Close the remaining implementation and demonstration risks identified after comparing the coverage report with the current repository.

Scope: Classification, governance, AI runtime, legacy rationalization, national codes, scalability, analytics, SAP integration, evaluation and end-to-end testing.

Prepared: 5 September 2026

Executive Summary

The core approval-to-national-code workflow is now present, together with duplicate detection, mapping, RBAC, import preview, mock SAP sync, data-quality metrics, demand aggregation and exports. The project is not yet feature-complete: automatic material classification is missing, while standardization, audit coverage, legacy conflict resolution, dashboard analytics, production AI reproducibility, scalability and real SAP connectivity remain partial.

Priority	Outcome	Items
P0	Credible and reliable live demo	Automatic classification, complete mutation audit, reproducible real AI engine, legacy conflict resolution
P1	Complete and scalable MVP	National-code structure, asynchronous imports, stronger analytics, real SAP connector architecture
P2	Defensible evidence and quality	Expanded model evaluation and full end-to-end tests

P0 1. Automatic Classification and Categorization

Current risk: Category is currently a passive field supplied by an uploaded row or user. The named requirement for intelligent classification and categorization is therefore not satisfied.

Implementation

- Create a classification service that consumes the normalized description and extracted engineering attributes.
- Start with transparent deterministic rules: BOLT/NUT/WASHER → FASTENERS; BALL/GATE VALVE → VALVES; TRANSFORMER/CABLE/MOTOR → ELECTRICAL.
- Return `category`, `subcategory`, `classification_confidence` and `classification_source` (RULE_ENGINE, ML or MANUAL).
- Invoke it during CSV/XLSX import, SAP sync and manual material creation whenever category is absent or untrusted.
- Allow a reviewer to correct the result, record the correction and use it as labelled data for a future ML classifier.
- Store category identifiers from a controlled taxonomy instead of relying only on free text.

```
classify_material(description, attributes) → {  
  "category": "FASTENERS",  
  "subcategory": "BOLTS",  
  "confidence": 0.96,  
  "source": "RULE_ENGINE"  
}
```

Acceptance criteria: An uploaded row without category is automatically assigned a category/subcategory; the confidence and rule/model version are visible; a reviewer can override it; the original and corrected values are audited.

2. Complete Audit Coverage

Current risk: Approvals and several request actions are audited, but imports and ordinary material mutations are not consistently covered. Governance claims therefore exceed the implementation.

Events that must be audited

- Material create, update, deactivate/delete and category correction.
- Import upload, preview, confirmation, completion and failure.
- SAP sync start, completion, failure and reconciliation.
- National-material creation, update and retirement.
- Mapping creation, reassignment and removal.
- AI recommendation generation, approval and rejection.
- User, role and permission changes.

For updates, store before/after values, actor, timestamp, correlation ID, source IP where appropriate, and a reason/comment. Prefer a reusable service or SQLAlchemy event mechanism so developers cannot accidentally omit logging.

Acceptance criteria: Every state-changing API operation creates an immutable audit event, and an auditor can filter events by actor, entity, action, CPSE and date.

3. Reproducible Real AI Runtime

Current risk: MiniLM/FAISS integration code exists, but the service can silently use the lexical fallback when dependencies, model weights or the teammate source path are unavailable.

Implementation

- Pin all Python and model versions.
- Download and cache the MiniLM model while building the AI Docker image; load it from a fixed container path.
- Set `TEAMMATE_AI_SRC=/teammate-ai/src` explicitly.
- Permit fallback only through an explicit development setting. In demo/production mode, fail startup if the real engine cannot load.
- Expose engine name, model/version, embedding dimension, FAISS availability and fallback reason in health output.
- Run AI unit and integration tests inside the same Docker image deployed for the demonstration.

```
AI_ENGINE_REQUIRED=true
ALLOW_FALLBACK=false
MODEL_PATH=/models/all-MiniLM-L6-v2
TEAMMATE_AI_SRC=/teammate-ai/src
```

Acceptance criteria: A clean Docker build starts with `TEAMMATE_REAL`, reports a loaded MiniLM model and FAISS index, passes a known-pair test, and never silently degrades when real AI is required.

4. Legacy Conflict Resolution

Current risk: Approval can create a national record from one source material, but no policy determines the canonical value when CPSE descriptions or attributes disagree.

Resolution policy

- Prefer complete, structured attributes over incomplete free text.
- Prefer recognized ISO, BIS, ASTM and API standards when evidence is present.
- Normalize equivalent units before deciding that values conflict.
- Never automatically resolve safety-critical differences such as grade, pressure rating, voltage or dimensions.
- Show conflicting values side-by-side and require reviewer selection or manual canonical entry.
- Preserve every original CPSE value and its provenance.

```
resolve_canonical(materials) → {  
  "proposed_description": "...",  
  "resolved_attributes": {...},  
  "conflicts": [{  
    "field": "material_grade",  
    "values": ["SS304", "SS316"],  
    "requires_review": true  
  }]  
}
```

Functional equivalents must not automatically share one national code unless a reviewer explicitly approves that governance policy.

Acceptance criteria: Conflicting critical attributes block automatic consolidation; a reviewer resolves each conflict; provenance and the final decision are auditable.

P1 5. Stronger National Material Code Scheme

Current risk: Sequential codes such as NMC-000001 work technically but provide no taxonomy or check-digit protection.

A proposed structure is:

NMC-FST-BLT-000123-7

```
graph LR
    A[NMC-FST-BLT-000123-7] --- B[check digit]
    A --- C[immutable sequence]
    A --- D[subcategory code]
    A --- E[category code]
```

- Maintain category/subcategory codes in controlled reference tables.
- Use a database sequence and unique constraint for concurrency safety.
- Compute a check digit to detect typing errors.
- Never change an issued national code when descriptions or classifications are corrected.
- Support aliases and retirement rather than reusing old codes.

Acceptance criteria: Codes are unique under concurrent creation, validate through a check digit, use controlled taxonomy values and remain immutable.

P1 6. Asynchronous, Scalable Imports and Matching

Current risk: Imports are synchronous and processed row-by-row. The real FAISS implementation uses a flat index and may rebuild candidate indexes, which is suitable for a demo but not national scale.

Upload → schema validation → create job → queue chunks
→ normalize/classify → bulk insert → candidate generation
→ reconciliation → completion summary

- Use Celery or RQ with Redis for the MVP; consider Kafka for larger deployment.
- Process chunks of approximately 500–2,000 rows and use database bulk inserts.
- Add idempotency keys, safe retry, cancellation, progress reporting and a failed-job/dead-letter path.
- Enforce file-size, row-count and content limits.
- Persist the vector index instead of rebuilding it for each request.
- At larger scale, replace IndexFlatIP with an approximate HNSW or IVF index and monitor recall.

Acceptance criteria: Large imports return immediately with a job ID, progress is observable, retries do not duplicate rows, failed chunks are recoverable, and the API remains responsive during processing.

P1 7. Complete Dashboard Analytics

Add backend-computed, filterable metrics rather than calculating business measures only in the browser:

- Duplicate-risk percentage and counts by Exact, Near-Duplicate and Functional-Equivalent.
- Mapping coverage and data-quality score by CPSE, sector and category.
- Demand aggregated by national code, with shortage and excess-inventory views.
- Estimated savings opportunity, approval turnaround time and import error rate.

- Drill-down from every metric to the underlying records.

An initial savings estimate may use `reusable surplus quantity × recent average unit price`. Clearly label it as an estimate and expose assumptions.

Acceptance criteria: Metrics are generated by documented backend queries, respond to filters, reconcile with source records and include duplicate detection plus demand information on the main dashboard.

P1 8. Real SAP/ERP Connector Architecture

Current risk: The mock SAP path demonstrates the flow but is not live enterprise connectivity.

```
class ERPConnector:
    def fetch_materials(...)
    def push_national_mapping(...)
    def health_check(...)
```

Implement separate `MockSAPConnector`, `SAP0DataConnector` and, if required, `SAPRFCConnector` adapters. Real connectivity requires:

- OAuth/client-certificate authentication and managed secrets.
- Pagination, delta tokens or last-modified incremental synchronization.
- Timeouts, exponential-backoff retry, rate limiting and circuit breaking.
- Versioned SAP-to-canonical field mappings.
- Idempotency, reconciliation and push-back of approved national codes.
- Complete integration audit records without logging credentials or sensitive payloads.

Acceptance criteria: The same canonical pipeline accepts mock and real connectors; incremental sync and retry are safe; approved mappings can be pushed back; every integration run is reconcilable and audited.

P2 9. Defensible Accuracy Evaluation

Current risk: “139/139” is a useful development result but is too small and context-free to support a general 100% accuracy claim.

Required evidence

- Dataset size, source, licensing and distribution by CPSE and material family.
- Separate tuning and held-out test sets, including previously unseen CPSE descriptions.
- Precision, recall and F1 for each class, confusion matrix and confidence calibration.
- Hard negatives: similar wording with different dimensions, pressure, voltage or grade.
- Missing attributes, unit conversions, spelling/OCR errors and multilingual descriptions.
- Model name, weights, rules, thresholds, code commit and evaluation date.

Run evaluation in CI and store a versioned JSON/HTML artifact. Present the result as “100% on the current 139-pair synthetic test set” until a larger independent set is available.

Acceptance criteria: A clean environment can reproduce the published metrics from a versioned dataset and model, and the report includes per-class performance plus failure examples.

P2 10. End-to-End Workflow Tests

The primary automated scenario must prove the complete product promise:

```
Import material
→ normalize and classify
→ generate AI candidate
→ reviewer compares and approves
→ NationalMaterial created
→ legacy materials mapped
→ audit events written
→ dashboard metrics updated
→ CSV/XLSX export contains mapping
```

Also cover rejection, duplicate import, unauthorized approval, maker-checker restrictions, conflicting safety-critical attributes, SAP failure/retry, unavailable AI engine and concurrent national-code generation.

Acceptance criteria: Tests run in CI against production-like PostgreSQL and service containers, fail on broken cross-service contracts, and leave repeatable test evidence.

Recommended Delivery Sequence

#	Work item	Why it comes here
1	Audit all state-changing operations	Establishes governance and makes later changes traceable.
2	Stabilize the real AI Docker runtime	Ensures the demonstrated engine matches the claimed engine.
3	Add rule-based automatic classification	Closes the only currently missing core requirement.
4	Add conflict-resolution review	Prevents unsafe canonical consolidation.

#	Work item	Why it comes here
5	Add end-to-end tests	Locks down the central One Nation, One Code workflow.
6	Improve the national-code format	Gives the registry a stable, defensible coding policy.
7	Add complete dashboard metrics	Closes the explicit analytics requirement.
8	Introduce asynchronous imports	Removes the largest operational scaling bottleneck.
9	Add a real SAP connector	Moves integration from simulation to enterprise readiness.
10	Expand independent AI evaluation	Makes accuracy claims credible and repeatable.

Definition of Done

- All 11 problem-statement requirements are demonstrably implemented or accurately labelled as simulated.
- The real AI engine and its model version are visible and reproducible.
- Automatic classification works and supports human correction.
- Critical legacy conflicts require explicit reviewer resolution.
- Every important mutation is audit logged with actor and before/after context.
- Dashboard analytics include duplicate risk, quality, mapping, demand and savings assumptions.
- Large imports are asynchronous, observable, retry-safe and idempotent.
- SAP wording clearly distinguishes mock demonstration from live connectivity.
- End-to-end tests prove approval → national code → mapping → audit → dashboard → export.
- Accuracy results are reproducible and qualified by dataset scope.